Compte rendu arbre

Taille et hauteur d'un arbre :

```python
def taille(self, t):
    if self.est_vide(t):
        return 0
    else:
        taille_gauche = self.taille(t[1])
        taille_droite = self.taille(t[2])
        return 1 + taille_gauche + taille_droite

def hauteur(self, t):
    if self.est_vide(t):
        return -1
    else:
        hauteur_gauche = self.hauteur(t[1])
        hauteur_droite = self.hauteur(t[2])
        return 1 + max(hauteur_gauche, hauteur_droite)
```

Methode est_vide :

```python
def est_vide(self, t):
    if t == [] :
        return True
    else:
        return False
```

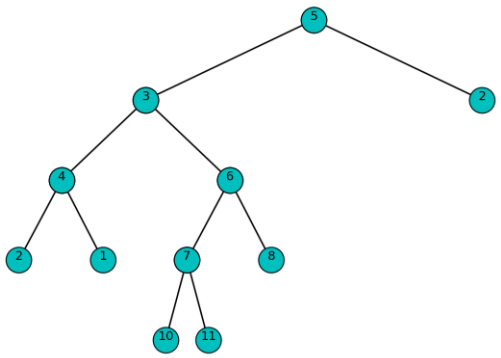
Contenu de main.py pour tester a, b, c, d et e :

```python
from PileFile import PileFile
from arbre_eleve import arbre
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = (20, 6)

if __name__ == "__main__":
    a = [5,[3,[4,[2,[],[]],[1,[],[]]],[6,[7,[10,[],[]],[11,[],[]]],[8,[],[]]]],[2,[],[]]]
    b = [1,[2,[4,[8,[],[]],[9,[],[]]],[5,[10,[],[]],[]]],[3,[6,[],[]],[7,[],[]]]]
    c = [2, [1, [0, [], []], []], [5, [4, [3, [], []], []], [12,[9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]],[]], [19, [], []]]]]]]]
    d = [20,[5,[3,[],[]],[12,[8,[6,[],[]],[]],[13,[],[]]]],[25,[21,[],[]],[28,[],[]]]]
    e = ["moi", ["père", ["grand-père_paternel", [], []], ["grand-mère_paternelle", [], []]], ["mère", ["grand-père_maternel", [], []], ["grand-mère_maternelle", [], []]]]


    arbre = arbre()

    noms = ["a", "b", "c", "d", "e"]
    arbres = [a, b, c, d, e]



    for i in range(len(noms)):
        print("Arbre " + noms[i] + ":")
        print("- Taille:", arbre.taille(arbres[i]))
        print("- Hauteur:", arbre.hauteur(arbres[i]))
        print("- Est vide:", arbre.est_vide(arbres[i]))
        print()
        arbre.dessiner(arbres[i])
```

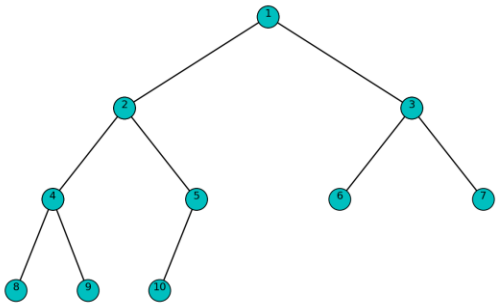
Affichage de la console :

```
Console ×

>>> %Run main.py

 Arbre a:
 - Taille: 11
 - Hauteur: 4
 - Est vide: False

 Arbre b:
 - Taille: 10
 - Hauteur: 3
 - Est vide: False

 Arbre c:
 - Taille: 20
 - Hauteur: 8
 - Est vide: False

 Arbre d:
 - Taille: 10
 - Hauteur: 4
 - Est vide: False

 Arbre e:
 - Taille: 7
 - Hauteur: 2
 - Est vide: False

>>> |
```

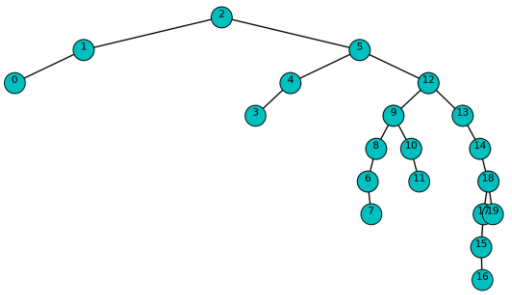
Représentation de a :



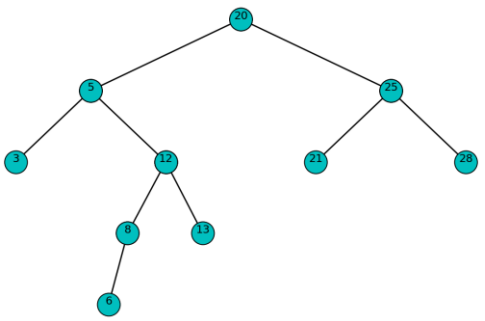
Représentation de b :



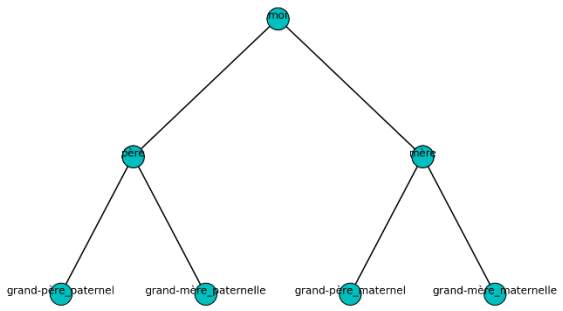
Représentation de c :



Représentation de d :



Représentation de e :



Méthode parcours_largeur :

```python
def parcours_largeur(self, t):
    visite = PileFile()
    lst = []
    visite.enfiler(t)
    while not visite.estVide():
        visite.afficher()
        noeud = visite.defiler()
        if type(noeud) == list and noeud != []:
            lst.append(noeud[0])
            if noeud[1] != []:
                visite.enfiler(noeud[1])
            if noeud[2] != []:
                visite.enfiler(noeud[2])
        else:
            lst.append(noeud)
    return lst
```

Tests (contenu de main.py):

```python
arbre = arbre()

noms = ["a", "b", "c", "d", "e"]
arbres = [a, b, c, d, e]


for i in range(len(noms)):
    print("Arbre " + noms[i] + ":")
#    print("- Taille:", arbre.taille(arbres[i]))
#    print("- Hauteur:", arbre.hauteur(arbres[i]))
#    print("- Est vide:", arbre.est_vide(arbres[i]))
#    print()
#    arbre.dessiner(arbres[i])

    print('Parcours par largeur :', arbre.parcours_largeur(arbres[i]))
    print()
```

Affichage de la console :

```
Arbre a:
[[5, [3, [4, [2, [], []], [1, [], []]], [6, [7, [10, [], []], [11, [], []]], [8, [], []]]], [2, [], []]]]
[[3, [4, [2, [], []], [1, [], []]], [6, [7, [10, [], []], [11, [], []]], [8, [], []]]], [2, [], []]]
[[2, [], []], [4, [2, [], []], [1, [], []]], [6, [7, [10, [], []], [11, [], []]], [8, [], []]]]
[[4, [2, [], []], [1, [], []]], [6, [7, [10, [], []], [11, [], []]], [8, [], []]]]
[[6, [7, [10, [], []], [11, [], []]], [8, [], []]], [2, [], []], [1, [], []]]
[[2, [], []], [1, [], []], [7, [10, [], []], [11, [], []]], [8, [], []]]
[[1, [], []], [7, [10, [], []], [11, [], []]], [8, [], []]]
[[7, [10, [], []], [11, [], []]], [8, [], []]]
[[8, [], []], [10, [], []], [11, [], []]]
[[10, [], []], [11, [], []]]
[[11, [], []]]
Parcours par largeur : [5, 3, 2, 4, 6, 2, 1, 7, 8, 10, 11]

Arbre b:
[[1, [2, [4, [8, [], []], [9, [], []]], [5, [10, [], []], []]], [3, [6, [], []], [7, [], []]]]]
[[2, [4, [8, [], []], [9, [], []]], [5, [10, [], []], []]], [3, [6, [], []], [7, [], []]]]
[[3, [6, [], []], [7, [], []]], [4, [8, [], []], [9, [], []]], [5, [10, [], []], []]]
[[4, [8, [], []], [9, [], []]], [5, [10, [], []], []], [6, [], []], [7, [], []]]
[[5, [10, [], []], []], [6, [], []], [7, [], []], [8, [], []], [9, [], []]]
[[6, [], []], [7, [], []], [8, [], []], [9, [], []], [10, [], []]]
[[7, [], []], [8, [], []], [9, [], []], [10, [], []]]
[[8, [], []], [9, [], []], [10, [], []]]
[[9, [], []], [10, [], []]]
[[10, [], []]]
Parcours par largeur : [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Arbre c:
[[2, [1, [0, [], []], []], [5, [4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [],
[]]]]]]]]]
[[1, [0, [], []], []], [5, [4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]
]]]]]]
[[5, [4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]], [0, [], []]]
[[0, [], []], [4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]]
[[4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]]
[[12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]], [3, [], []]]
[[3, [], []], [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]
[[9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]
[[13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]], [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]]
[[8, [6, [], [7, [], []]], []], [10, [], [11, [], []]], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]
[[10, [], [11, [], []]], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]], [6, [], [7, [], []]]]
[[14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]], [6, [], [7, [], []]], [11, [], []]]
[[6, [], [7, [], []]], [11, [], []]], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]
[[11, [], []], [18, [17, [15, [], [16, [], []]], []], [19, [], []]], [7, [], []]]
[[18, [17, [15, [], [16, [], []]], []], [19, [], []]], [7, [], []]]
[[7, [], []], [17, [15, [], [16, [], []]], []], [19, [], []]]
[[17, [15, [], [16, [], []]], []], [19, [], []]]
[[19, [], []], [15, [], [16, [], []]]]
[[15, [], [16, [], []]]]
[[16, [], []]]
Parcours par largeur : [2, 1, 5, 0, 4, 12, 3, 9, 13, 8, 10, 14, 6, 11, 18, 7, 17, 19, 15, 16]

Arbre d:
[[20, [5, [3, [], []], []], [12, [8, [6, [], []], []], [13, [], []]]], [25, [21, [], []], [28, [], []]]]
[[5, [3, [], []], []], [12, [8, [6, [], []], []], [13, [], []]], [25, [21, [], []], [28, [], []]]]
[[25, [21, [], []], [28, [], []]], [3, [], []], [12, [8, [6, [], []], []], [13, [], []]]]
[[3, [], []], [12, [8, [6, [], []], []], [13, [], []]], [21, [], []], [28, [], []]]
[[12, [8, [6, [], []], []], [13, [], []]], [21, [], []], [28, [], []]]
[[21, [], []], [28, [], []], [8, [6, [], []], []], [13, [], []]]
[[28, [], []], [8, [6, [], []], []], [13, [], []]]
[[8, [6, [], []], []], [13, [], []]]
[[13, [], []], [6, [], []]]
[[6, [], []]]
Parcours par largeur : [20, 5, 25, 3, 12, 21, 28, 8, 13, 6]
```

```
Arbre e:
[['moi', ['père', ['grand-père_paternel', [], []], ['grand-mère_paternelle', [], []]], ['mère', ['grand-père_maternel', [], []], ['grand-mère_maternelle', [], []]]]]
[['père', ['grand-père_paternel', [], []], ['grand-mère_paternelle', [], []]], ['mère', ['grand-père_maternel', [], []], ['grand-mère_maternelle', [], []]]]
[['mère', ['grand-père_maternel', [], []], ['grand-mère_maternelle', [], []]], ['grand-père_paternel', [], []], ['grand-mère_paternelle', [], []]]
[['grand-père_paternel', [], []], ['grand-mère_paternelle', [], []], ['grand-père_maternel', [], []], ['grand-mère_maternelle', [], []]]
[['grand-mère_paternelle', [], []], ['grand-père_maternel', [], []], ['grand-mère_maternelle', [], []]]
[['grand-père_maternel', [], []], ['grand-mère_maternelle', [], []]]
[['grand-mère_maternelle', [], []]]
Parcours par largeur : ['moi', 'père', 'mère', 'grand-père_paternel', 'grand-mère_paternelle', 'grand-père_maternel', 'grand-mère_maternelle']
```

Methode parcours_profondeur :

```python
def parcours_profondeur(self, t):
    visite = PileFile()
    lst1 = []
    visite.empiler(t)
    while not visite.estVide():
        visite.afficher()
        noeud = visite.depiler()
        if type(noeud) == list and noeud != []:
            lst1.append(noeud[0])
            if noeud[2] != []:
                visite.empiler(noeud[2])
            if noeud[1] != []:
                visite.empiler(noeud[1])
        else:
            lst1.append(noeud)
    return lst1
```

Tests(contenu de main.py) :

```python
    for i in range(len(noms)):
        print("Arbre " + noms[i] + ":")
#        print("- Taille:", arbre.taille(arbres[i]))
#        print("- Hauteur:", arbre.hauteur(arbres[i]))
#        print("- Est vide:", arbre.est_vide(arbres[i]))
#        print()
#        arbre.dessiner(arbres[i])

#        print('Parcours par largeur :', arbre.parcours_largeur(arbres[i]))
#        print()

        print('Parcours par profondeur :', arbre.parcours_profondeur(arbres[i]))
        print()
```

Affichage de la console :

```
Arbre a:
[[5, [3, [4, [2, [], []], [1, [], []]], [6, [7, [10, [], []], [11, [], []]], [8, [], []]]], [2, [], []]]]
[[2, [], []], [3, [4, [2, [], []], [1, [], []]], [6, [7, [10, [], []], [11, [], []]], [8, [], []]]]]
[[2, [], []], [6, [7, [10, [], []], [11, [], []]], [8, [], []]], [4, [2, [], []], [1, [], []]]]
[[2, [], []], [6, [7, [10, [], []], [11, [], []]], [8, [], []]], [1, [], []], [2, [], []]]
[[2, [], []], [6, [7, [10, [], []], [11, [], []]], [8, [], []]], [1, [], []]]
[[2, [], []], [6, [7, [10, [], []], [11, [], []]], [8, [], []]]]
[[2, [], []], [8, [], []], [7, [10, [], []], [11, [], []]]]
[[2, [], []], [8, [], []], [11, [], []], [10, [], []]]
[[2, [], []], [8, [], []], [11, [], []]]
[[2, [], []], [8, [], []]]
[[2, [], []]]
Parcours par profondeur : [5, 3, 4, 2, 1, 6, 7, 10, 11, 8, 2]

Arbre b:
[[1, [2, [4, [8, [], []], [9, [], []]], [5, [10, [], []], []]], [3, [6, [], []], [7, [], []]]]]
[[3, [6, [], []], [7, [], []]], [2, [4, [8, [], []], [9, [], []]], [5, [10, [], []], []]]]
[[3, [6, [], []], [7, [], []]], [5, [10, [], []], []], [4, [8, [], []], [9, [], []]]]
[[3, [6, [], []], [7, [], []]], [5, [10, [], []], []], [9, [], []], [8, [], []]]
[[3, [6, [], []], [7, [], []]], [5, [10, [], []], []], [9, [], []]]
[[3, [6, [], []], [7, [], []]], [5, [10, [], []], []]]
[[3, [6, [], []], [7, [], []]], [10, [], []]]
[[3, [6, [], []], [7, [], []]]]
[[7, [], []], [6, [], []]]
[[7, [], []]]
Parcours par profondeur : [1, 2, 4, 8, 9, 5, 10, 3, 6, 7]
```

```
Arbre c:
[[2, [1, [0, [], []], []], [5, [4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [],
[]]]]]]]]]
[[5, [4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]], [1, [0, [], []]
, []]]
[[5, [4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]], [0, [], []]]
[[5, [4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]]
[[12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]], [4, [3, [], []], []]]
[[12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]], [3, [], []]]
[[12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]
[[13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]], [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]]]
[[13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]], [10, [], [11, [], []]], [8, [6, [], [7, [], []]], []]]
[[13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]], [10, [], [11, [], []]], [6, [], [7, [], []]]]
[[13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]], [10, [], [11, [], []]], [7, [], []]]
[[13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]], [10, [], [11, [], []]]]
[[13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]], [11, [], []]]
[[13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]
[[14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]
[[18, [17, [15, [], [16, [], []]], []], [19, [], []]]]
[[19, [], []], [17, [15, [], [16, [], []]], []]]
[[19, [], []], [15, [], [16, [], []]]]
[[19, [], []], [16, [], []]]
[[19, [], []]]
Parcours par profondeur : [2, 1, 0, 5, 4, 3, 12, 9, 8, 6, 7, 10, 11, 13, 14, 18, 17, 15, 16, 19]

Arbre d:
[[20, [5, [3, [], []], [12, [8, [6, [], []], []], [13, [], []]]], [25, [21, [], []], [28, [], []]]]]
[[25, [21, [], []], [28, [], []]], [5, [3, [], []], [12, [8, [6, [], []], []], [13, [], []]]]]
[[25, [21, [], []], [28, [], []]], [12, [8, [6, [], []], []], [13, [], []]], [3, [], []]]
[[25, [21, [], []], [28, [], []]], [12, [8, [6, [], []], []], [13, [], []]]]
[[25, [21, [], []], [28, [], []]], [13, [], []], [8, [6, [], []], []]]
[[25, [21, [], []], [28, [], []]], [13, [], []], [6, [], []]]
[[25, [21, [], []], [28, [], []]], [13, [], []]]
[[25, [21, [], []], [28, [], []]]]
[[28, [], []], [21, [], []]]
[[28, [], []]]
Parcours par profondeur : [20, 5, 3, 12, 8, 6, 13, 25, 21, 28]


Arbre e:
[['moi', ['père', ['grand-père_paternel', [], []], ['grand-mère_paternelle', [], []]], ['mère', ['grand-père_maternel', [], []], ['grand-mère_maternelle', [], []]]]]
[['mère', ['grand-père_maternel', [], []], ['grand-mère_maternelle', [], []]], ['père', ['grand-père_paternel', [], []], ['grand-mère_paternelle', [], []]]]
[['mère', ['grand-père_maternel', [], []], ['grand-mère_maternelle', [], []]], ['grand-mère_paternelle', [], []], ['grand-père_paternel', [], []]]
[['mère', ['grand-père_maternel', [], []], ['grand-mère_maternelle', [], []]], ['grand-mère_paternelle', [], []]]
[['mère', ['grand-père_maternel', [], []], ['grand-mère_maternelle', [], []]]]
[['grand-mère_maternelle', [], []], ['grand-père_maternel', [], []]]
[['grand-mère_maternelle', [], []]]
Parcours par profondeur : ['moi', 'père', 'grand-père_paternel', 'grand-mère_paternelle', 'mère', 'grand-père_maternel', 'grand-mère_maternelle']
```

Méthode parcours_profondeur_recursif_prefixe :

```python
def parcours_profondeur_recursif_prefixe(self, t):
    lst2 = []
    if type(t) == list and t != []:
        lst2.append(t[0])
        if t[1] != []:
            print(t)
            lst2.extend(self.parcours_profondeur_recursif_prefixe(t[1]))
        if t[2] != []:
            print(t)
            lst2.extend(self.parcours_profondeur_recursif_prefixe(t[2]))
    else:
        print(t)
        lst2.append(t)
    return lst2
```

Contenu de main.py :

```python
    arbre = arbre()

    noms = ["a", "b", "c", "d", "e"]
    arbres = [a, b, c, d, e]


    for i in range(len(noms)):
        print("Arbre " + noms[i] + ":")
#        print("- Taille:", arbre.taille(arbres[i]))
#        print("- Hauteur:", arbre.hauteur(arbres[i]))
#        print("- Est vide:", arbre.est_vide(arbres[i]))
#        print()
#        arbre.dessiner(arbres[i])

#        print('Parcours par largeur :', arbre.parcours_largeur(arbres[i]))
#        print()

#        print('Parcours par profondeur :', arbre.parcours_profondeur(arbres[i]))
#        print()

        print('Parcours par profondeur récursif préfixe :', arbre.parcours_profondeur_recursif_prefixe(arbres[i]))
        print()
```

Affichage de la console :

```
Arbre a:
[5, [3, [4, [2, [], []], [1, [], []]], [6, [7, [10, [], []], [11, [], []]], [8, [], []]]], [2, [], []]]
[3, [4, [2, [], []], [1, [], []]], [6, [7, [10, [], []], [11, [], []]], [8, [], []]]]
[4, [2, [], []], [1, [], []]]
[4, [2, [], []], [1, [], []]]
[3, [4, [2, [], []], [1, [], []]], [6, [7, [10, [], []], [11, [], []]], [8, [], []]]]
[6, [7, [10, [], []], [11, [], []]], [8, [], []]]
[7, [10, [], []], [11, [], []]]
[7, [10, [], []], [11, [], []]]
[6, [7, [10, [], []], [11, [], []]], [8, [], []]]
[5, [3, [4, [2, [], []], [1, [], []]], [6, [7, [10, [], []], [11, [], []]], [8, [], []]]], [2, [], []]]
Parcours par profondeur récursif préfixe : [5, 3, 4, 2, 1, 6, 7, 10, 11, 8, 2]

Arbre b:
[1, [2, [4, [8, [], []], [9, [], []]], [5, [10, [], []], []]], [3, [6, [], []], [7, [], []]]]
[2, [4, [8, [], []], [9, [], []]], [5, [10, [], []], []]]
[4, [8, [], []], [9, [], []]]
[4, [8, [], []], [9, [], []]]
[2, [4, [8, [], []], [9, [], []]], [5, [10, [], []], []]]
[5, [10, [], []], []]
[1, [2, [4, [8, [], []], [9, [], []]], [5, [10, [], []], []]], [3, [6, [], []], [7, [], []]]]
[3, [6, [], []], [7, [], []]]
[3, [6, [], []], [7, [], []]]
Parcours par profondeur récursif préfixe : [1, 2, 4, 8, 9, 5, 10, 3, 6, 7]
```

```
Arbre c:
[2, [1, [0, [], []], []], [5, [4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], [
[]]]]]]]]
[1, [0, [], []], []]
[2, [1, [0, [], []], []], [5, [4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], [
[]]]]]]]]
[5, [4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]]
[4, [3, [], []], []]
[5, [4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]]
[12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]
[9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]]
[8, [6, [], [7, [], []]], []]
[6, [], [7, [], []]]
[9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]]
[10, [], [11, [], []]]
[12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]
[13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]
[14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]
[18, [17, [15, [], [16, [], []]], []], [19, [], []]]
[17, [15, [], [16, [], []]], []]
[15, [], [16, [], []]]
[18, [17, [15, [], [16, [], []]], []], [19, [], []]]
Parcours par profondeur récursif préfixe : [2, 1, 0, 5, 4, 3, 12, 9, 8, 6, 7, 10, 11, 13, 14, 18, 17, 15, 16, 19]

Arbre d:
[20, [5, [3, [], []], [12, [8, [6, [], []], []], [13, [], []]]], [25, [21, [], []], [28, [], []]]]
[5, [3, [], []], [12, [8, [6, [], []], []], [13, [], []]]]
[5, [3, [], []], [12, [8, [6, [], []], []], [13, [], []]]]
[12, [8, [6, [], []], []], [13, [], []]]
[8, [6, [], []], []]
[12, [8, [6, [], []], []], [13, [], []]]
[20, [5, [3, [], []], [12, [8, [6, [], []], []], [13, [], []]]], [25, [21, [], []], [28, [], []]]]
[25, [21, [], []], [28, [], []]]
[25, [21, [], []], [28, [], []]]
Parcours par profondeur récursif préfixe : [20, 5, 3, 12, 8, 6, 13, 25, 21, 28]
```

```
Arbre e:
['moi', ['père', ['grand-père_paternel', [], []], ['grand-mère_paternelle', [], []]], ['mère', ['grand-père_maternel', [], []], ['grand-mère_maternelle', [], []]]]
['père', ['grand-père_paternel', [], []], ['grand-mère_paternelle', [], []]]
['père', ['grand-père_paternel', [], []], ['grand-mère_paternelle', [], []]]
['moi', ['père', ['grand-père_paternel', [], []], ['grand-mère_paternelle', [], []]], ['mère', ['grand-père_maternel', [], []], ['grand-mère_maternelle', [], []]]]
['mère', ['grand-père_maternel', [], []], ['grand-mère_maternelle', [], []]]
['mère', ['grand-père_maternel', [], []], ['grand-mère_maternelle', [], []]]
Parcours par profondeur récursif préfixe : ['moi', 'père', 'grand-père_paternel', 'grand-mère_paternelle', 'mère', 'grand-père_maternel', 'grand-mère_maternelle']
```

Méthode parcours_profondeur_infixe :

```python
def parcours_profondeur_recursif_infixe(self, t):
    lst2 = []
    if type(t) == list and t != []:
        if t[1] != []:
            print(t)
            lst2.extend(self.parcours_profondeur_recursif_infixe(t[1]))
        lst2.append(t[0])
        if t[2] != []:
            print(t)
            lst2.extend(self.parcours_profondeur_recursif_infixe(t[2]))
    else:
        print(t)
        lst2.append(t)
    return lst2
```

Contenu de main.py :

```python
arbre = arbre()

noms = ["a", "b", "c", "d", "e"]
arbres = [a, b, c, d, e]


for i in range(len(noms)):
    print("Arbre " + noms[i] + ":")
     print("- Taille:", arbre.taille(arbres[i]))
     print("- Hauteur:", arbre.hauteur(arbres[i]))
     print("- Est vide:", arbre.est_vide(arbres[i]))
     print()
     arbre.dessiner(arbres[i])

     print('Parcours par largeur :', arbre.parcours_largeur(arbres[i]))
     print()

     print('Parcours par profondeur :', arbre.parcours_profondeur(arbres[i]))
     print()

     print('Parcours par profondeur récursif préfixe :', arbre.parcours_profondeur_recursif_prefixe(arbres[i]))
     print()

    print('Parcours par profondeur récursif infixe :', arbre.parcours_profondeur_recursif_infixe(arbres[i]))
    print()
```

Affichage de la console :

```
Arbre a:
[5, [3, [4, [2, [], []], [1, [], []]], [6, [7, [10, [], []], [11, [], []]], [8, [], []]]], [2, [], []]]
[3, [4, [2, [], []], [1, [], []]], [6, [7, [10, [], []], [11, [], []]], [8, [], []]]]
[4, [2, [], []], [1, [], []]]
[4, [2, [], []], [1, [], []]]
[3, [4, [2, [], []], [1, [], []]], [6, [7, [10, [], []], [11, [], []]], [8, [], []]]]
[6, [7, [10, [], []], [11, [], []]], [8, [], []]]
[7, [10, [], []], [11, [], []]]
[7, [10, [], []], [11, [], []]]
[6, [7, [10, [], []], [11, [], []]], [8, [], []]]
[5, [3, [4, [2, [], []], [1, [], []]], [6, [7, [10, [], []], [11, [], []]], [8, [], []]]], [2, [], []]]
Parcours par profondeur récursif infixe : [2, 4, 1, 3, 10, 7, 11, 6, 8, 5, 2]

Arbre b:
[1, [2, [4, [8, [], []], [9, [], []]], [5, [10, [], []], []]], [3, [6, [], []], [7, [], []]]]
[2, [4, [8, [], []], [9, [], []]], [5, [10, [], []], []]]
[4, [8, [], []], [9, [], []]]
[4, [8, [], []], [9, [], []]]
[2, [4, [8, [], []], [9, [], []]], [5, [10, [], []], []]]
[5, [10, [], []], []]
[1, [2, [4, [8, [], []], [9, [], []]], [5, [10, [], []], []]], [3, [6, [], []], [7, [], []]]]
[3, [6, [], []], [7, [], []]]
[3, [6, [], []], [7, [], []]]
Parcours par profondeur récursif infixe : [8, 4, 9, 2, 10, 5, 1, 6, 3, 7]
```

```
Arbre c:
[2, [1, [0, [], []], []], [5, [4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]
], []], [19, [], []]]]]]]]
[1, [0, [], []], []]
[2, [1, [0, [], []], []], [5, [4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]
], []], [19, [], []]]]]]]]
[5, [4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]]]
[4, [3, [], []], []]
[5, [4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]]]
[12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]
[9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]]
[8, [6, [], [7, [], []]], []]
[6, [], [7, [], []]]
[9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]]
[10, [], [11, [], []]]
[12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]
[13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]
[14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]
[18, [17, [15, [], [16, [], []]], []], [19, [], []]]
[17, [15, [], [16, [], []]], []]
[15, [], [16, [], []]]
[18, [17, [15, [], [16, [], []]], []], [19, [], []]]
Parcours par profondeur récursif infixe : [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]

Arbre d:
[20, [5, [3, [], []], [12, [8, [6, [], []], []], [13, [], []]]], [25, [21, [], []], [28, [], []]]]
[5, [3, [], []], [12, [8, [6, [], []], []], [13, [], []]]]
[5, [3, [], []], [12, [8, [6, [], []], []], [13, [], []]]]
[12, [8, [6, [], []], []], [13, [], []]]
[8, [6, [], []], []]
[12, [8, [6, [], []], []], [13, [], []]]
[20, [5, [3, [], []], [12, [8, [6, [], []], []], [13, [], []]]], [25, [21, [], []], [28, [], []]]]
[25, [21, [], []], [28, [], []]]
[25, [21, [], []], [28, [], []]]
Parcours par profondeur récursif infixe : [3, 5, 6, 8, 12, 13, 20, 21, 25, 28]
```

```
Arbre e:
['moi', ['père', ['grand-père_paternel', [], []], ['grand-mère_paternelle', [], []]], ['mère', ['grand-père_maternel', [], []], ['grand-mère_maternelle', [], []]]]
['père', ['grand-père_paternel', [], []], ['grand-mère_paternelle', [], []]]
['père', ['grand-père_paternel', [], []], ['grand-mère_paternelle', [], []]]
['moi', ['père', ['grand-père_paternel', [], []], ['grand-mère_paternelle', [], []]], ['mère', ['grand-père_maternel', [], []], ['grand-mère_maternelle', [], []]]]
['mère', ['grand-père_maternel', [], []], ['grand-mère_maternelle', [], []]]
['mère', ['grand-père_maternel', [], []], ['grand-mère_maternelle', [], []]]
Parcours par profondeur récursif infixe : ['grand-père_paternel', 'père', 'grand-mère_paternelle', 'moi', 'grand-père_maternel', 'mère', 'grand-mère_maternelle']
```

methode parcours_profondeur_recursif_postfixe :

```python
def parcours_profondeur_recursif_postfixe(self, t):
    lst2 = []
    if type(t) == list and t != []:
        if t[1] != []:
            print(t)
            lst2.extend(self.parcours_profondeur_recursif_postfixe(t[1]))
        if t[2] != []:
            print(t)
            lst2.extend(self.parcours_profondeur_recursif_postfixe(t[2]))
        lst2.append(t[0])
    else:
        print(t)
        lst2.append(t)
    return lst2
```

contenu de main.py :

```python
arbres = [a, b, c, d, e]


for i in range(len(noms)):
    print("Arbre " + noms[i] + ":")
#       print("- Taille:", arbre.taille(arbres[i]))
#       print("- Hauteur:", arbre.hauteur(arbres[i]))
#       print("- Est vide:", arbre.est_vide(arbres[i]))
#       print()
#       arbre.dessiner(arbres[i])

#       print('Parcours par largeur :', arbre.parcours_largeur(arbres[i]))
#       print()

#       print('Parcours par profondeur :', arbre.parcours_profondeur(arbres[i]))
#       print()

#       print('Parcours par profondeur récursif préfixe :', arbre.parcours_profondeur_recursif_prefixe(arbres[i]))
#       print()

#       print('Parcours par profondeur récursif infixe :', arbre.parcours_profondeur_recursif_infixe(arbres[i]))
#       print()

    print('Parcours par profondeur récursif postfixe :', arbre.parcours_profondeur_recursif_postfixe(arbres[i]))
    print()
```

Affichage de la console :

```
Arbre a:
[5, [3, [4, [2, [], []], [1, [], []]], [6, [7, [10, [], []], [11, [], []]], [8, [], []]]], [2, [], []]]
[3, [4, [2, [], []], [1, [], []]], [6, [7, [10, [], []], [11, [], []]], [8, [], []]]]
[4, [2, [], []], [1, [], []]]
[4, [2, [], []], [1, [], []]]
[3, [4, [2, [], []], [1, [], []]], [6, [7, [10, [], []], [11, [], []]], [8, [], []]]]
[6, [7, [10, [], []], [11, [], []]], [8, [], []]]
[7, [10, [], []], [11, [], []]]
[7, [10, [], []], [11, [], []]]
[6, [7, [10, [], []], [11, [], []]], [8, [], []]]
[5, [3, [4, [2, [], []], [1, [], []]], [6, [7, [10, [], []], [11, [], []]], [8, [], []]]], [2, [], []]]
Parcours par profondeur récursif infixe : [2, 4, 1, 3, 10, 7, 11, 6, 8, 5, 2]

Arbre b:
[1, [2, [4, [8, [], []], [9, [], []]], [5, [10, [], []], []]], [3, [6, [], []], [7, [], []]]]
[2, [4, [8, [], []], [9, [], []]], [5, [10, [], []], []]]
[4, [8, [], []], [9, [], []]]
[4, [8, [], []], [9, [], []]]
[2, [4, [8, [], []], [9, [], []]], [5, [10, [], []], []]]
[5, [10, [], []], []]
[1, [2, [4, [8, [], []], [9, [], []]], [5, [10, [], []], []]], [3, [6, [], []], [7, [], []]]]
[3, [6, [], []], [7, [], []]]
[3, [6, [], []], [7, [], []]]
Parcours par profondeur récursif infixe : [8, 4, 9, 2, 10, 5, 1, 6, 3, 7]
```

```
Arbre c:
[2, [1, [0, [], []], []], [5, [4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]
], []], [19, [], []]]]]]]]
[1, [0, [], []], []]
[2, [1, [0, [], []], []], [5, [4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]
], []], [19, [], []]]]]]]]
[5, [4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]]
[4, [3, [], []], []]
[5, [4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]]
[12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]
[9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]]
[8, [6, [], [7, [], []]], []]
[6, [], [7, [], []]]
[9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]]
[10, [], [11, [], []]]
[12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]
[13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]
[14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]
[18, [17, [15, [], [16, [], []]], []], [19, [], []]]
[17, [15, [], [16, [], []]], []]
[15, [], [16, [], []]]
[18, [17, [15, [], [16, [], []]], []], [19, [], []]]
Parcours par profondeur récursif infixe : [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]

Arbre d:
[20, [5, [3, [], []], [12, [8, [6, [], []], []], [13, [], []]]], [25, [21, [], []], [28, [], []]]]
[5, [3, [], []], [12, [8, [6, [], []], []], [13, [], []]]]
[5, [3, [], []], [12, [8, [6, [], []], []], [13, [], []]]]
[12, [8, [6, [], []], []], [13, [], []]]
[8, [6, [], []], []]
[12, [8, [6, [], []], []], [13, [], []]]
[20, [5, [3, [], []], [12, [8, [6, [], []], []], [13, [], []]]], [25, [21, [], []], [28, [], []]]]
[25, [21, [], []], [28, [], []]]
[25, [21, [], []], [28, [], []]]
Parcours par profondeur récursif infixe : [3, 5, 6, 8, 12, 13, 20, 21, 25, 28]

  Arbre e:
  ['moi', ['père', ['grand-père_paternel', [], []], ['grand-mère_paternelle', [], []]], ['mère', ['grand-père_maternel', [], []], ['grand-mère_maternelle', [], []]]]
  ['père', ['grand-père_paternel', [], []], ['grand-mère_paternelle', [], []]]
  ['père', ['grand-père_paternel', [], []], ['grand-mère_paternelle', [], []]]
  ['moi', ['père', ['grand-père_paternel', [], []], ['grand-mère_paternelle', [], []]], ['mère', ['grand-père_maternel', [], []], ['grand-mère_maternelle', [], []]]]
  ['mère', ['grand-père_maternel', [], []], ['grand-mère_maternelle', [], []]]
  ['mère', ['grand-père_maternel', [], []], ['grand-mère_maternelle', [], []]]
  Parcours par profondeur récursif infixe : ['grand-père_paternel', 'père', 'grand-mère_paternelle', 'moi', 'grand-père_maternel', 'mère', 'grand-mère_maternelle']

>>>
```